System software (Operating systems)
Laboratory work №3
Page replacement algorithms

Virtual memory is a software-hardware memory management technology. One of the tasks of virtual memory is to provide programs with memory address spaces that exceed the size of physical memory. Virtual memory consists of three components: virtual address space, physical address space and a mechanism for translation or mapping of virtual addresses to physical addresses.

Virtual address space is a set of memory addresses that are involved in processor instructions that work with program memory. These addresses are called *virtual addresses*. *Physical address space* is a set of random access memory (RAM) addresses. These addresses are called *physical addresses*. Virtual addresses are translated into physical addresses by the *memory management unit* (MMU) using a *mapping mechanism* defined by the computer architecture. Mapping mechanisms can be different, but they all connect the current virtual address space (a memory space that does not physically correspond to the memory of any computer device, it is an abstraction) with a physical address space (with RAM, that is the physical computer device).

Virtual and physical address spaces are usually flat and consist of *virtual* and *physical pages*, respectively (a page is an array of bytes). The address of the beginning of the page must be aligned to its size. The mapping mechanism allows to define the properties of each virtual page, these properties are called *virtual page attributes*. The most important attributes are: the presence bit, the bit that determines permission to modify the page contents, the number of the physical page to which the virtual page is mapped. The following attributes may also be supported: reference bit and modification bit. A virtual page can be mapped to a single physical page or to an array of multiple physical pages, the size of the virtual pages and the size of the physical page are determined by the computer architecture.

One of the mapping mechanisms is a *page table*. Each process has its own page table. The page table consists of *page table entries* (PTE) that contain attributes of virtual pages.

Attributes of virtual pages that are relevant to the laboratory work:

- P, presence bit. The MMU checks this bit each time a virtual page is accessed. If this bit is set, then all other attributes of this PTE are meaningful. If this bit is not set, then the *page fault* exception is generated on attempt to access this virtual page. The kernel sets this bit when creating a virtual page mapping and clears this bit when destroying the mapping.
- R, reference bit, or A, access bit. The MMU sets this bit each time the page contents are accessed. The kernel clears this bit to then check if it has been set, thus determining if the page contents have been accessed.
- M, modification bit, or D, dirty bit. The MMU sets this bit when modifying the page contents. The kernel can check this bit to determine if the page contents have been modified. A modified page is called *dirty*. The kernel can save the modified page contents in the corresponding FS object and clear this bit. An unmodified page is called *clean*.
- PPN, physical page number. The MMU uses this number when translating a virtual address to a physical address. The kernel sets this number when creating the virtual page mapping.

A *page fault* is an exception that occurs when accessing a non-mapped virtual page. As a result of generating a page fault, the system changes the execution mode to kernel mode, the corresponding kernel

function handles this exception. The task of this kernel function is to find a free physical page or free an occupied physical page. Searching for a physical page for replacement is the responsibility of the kernel function, which (fully or partially) implements the *page replacement algorithm*.

The difficulty of making a decision about choosing a physical page for replacement is caused by the fact that there is no information about subsequent memory accesses by processes. However, effective page replacement algorithms can be created, since programs that solve real tasks have the characteristic of *locality of references*: in a certain time interval, any process accesses a limited set of pages from its virtual address space, this a set of pages is called a *working set*, later the working set is changed completely or partially. Such a working set exists because a program that solves a real task works according to some algorithm in which there are no accesses to all or random memory addresses for some time interval (in a program, this applies to memory addresses for code and data). In this way, the program has a permanent set of virtual pages that it accesses more often than others. If the program does not solve a real task and specifically accesses random addresses in its memory, then such a program does not have a working set.

The task of any page replacement algorithm is to predict the current working set of virtual pages (the aggregate working set of all processes). This prediction is implemented by checking the attribute values of all mapped virtual pages of all processes at certain time intervals. Virtual pages that belong to the working set are called *active*, other virtual pages are called *inactive*. First of all, the most inactive pages are replaced: their contents are moved to the corresponding FS objects, their current mappings are removed, the “freed” physical pages are used to create mappings of virtual pages of those processes that need to create mappings of their virtual pages to continue their execution. If there are no inactive pages, then active pages are replaced, which is inefficient for the system, since they are most likely to be accessed in the near future.

Initial conditions

- There are one processor and N physical pages in the system.
- There can be multiple ready-to-run processes in the system. Processes are created and terminated. The maximum number of processes is limited.
- Processes get CPU time in turn.
- Each process has its own virtual address space (its own page table), the number of pages in which can be less or more than N. The page table is a one-dimensional array. A virtual page can be mapped to only one physical page. One physical page can be used in only one mapping.
- When creating a new process, all its virtual pages are not mapped (demand paging).
- Each process accesses its virtual pages with the characteristic of locality of references (90% of accesses are to virtual pages from the working set, 10% of accesses are to any virtual pages). The request can be read or modified (choose the proportion of request types yourself). The working set of the process must be changed over time.
- The system uses a page replacement algorithm. Mappings of virtual pages are created on demand (demand paging). The system has a timer and additional system processes can be run if necessary. For example, it can be a background system process that checks the attribute values of virtual pages.
- A global policy in a page replacement algorithm is used. The global policy means that processes compete equally for all physical memory.
- The virtual page attributes are only: the presence bit, the reference bit, the modification bit, and the physical page number. The system has a FS where the contents of pages can be written, it is

not necessary to implement the FS, but it is necessary to distinguish the pages whose contents were written in the FS from the pages that have not yet been accessed.
- The execution time of a process, the time when to create a new process, the time quantum, the time to change the working set are determined by the number of page accesses.

The task

- Develop a model (program) of a system with a paged memory that corresponds to the above specified initial conditions, with the implementation of two page replacement algorithms according to the variant.
- The program should output to the standard output stream (or to a file) a report on the characteristics of virtual memory of processes, page faults, the decisions of the page replacement algorithm, etc.
- Page replacement algorithms are chosen according to the variant (the variant number is determined by the number of the gradebook, last_two_digits mod 3 + 1).
- By changing the size of the working set, check the correctness of the model implementation and the implemented page replacement algorithm. If the number of virtual pages in the working set is increased, then the frequency of page faults should increase. The random page replacament algorithm should result in a higher page fault rate than the other page replacement algorithm from the variant.
- It is desirable to distinguish the following classes (structures) in the program: process, kernel, MMU, physical page, page table, working set.

Variants

1. Random page replacement and NRU (Not Recently Used) algorithms.
2. Random page replacement and Clock algorithms.
3. Random page replacement and WSClock (Working Set Clock) algorithms.

Report

- Descriptions of page replacement algorithms (optional).
- Description of the developed program (optional).
- Listing of the developed program.

Literature

- Andrew S. Tanenbaum, Herbert Bos, *Modern Operating Systems*, 4th edition.